ADS205 Decision, Computation and Language

Lecture 4 – Context-Free Language

Dr Chunchuan Lyu and Dr Xinhong Chen



Xi'an Jiaotong-Liverpool University
西交利物浦大学

Recap

- Regular Expression
- Regexes ↔ FAs
- The Pumping Lemma for regular languages

Today

- Context-Free Grammars
- Context-Free Languages



Recap: Regular Languages & Proving Non-regularity

- Regular Expression are equivalent to FAs.
- Proving regularity means writing regexes or constructing FAs for a language.
- Proving non-regularity means using the regular pumping lemma and find contradiction.
- For regular languages, we have Regexes, FAs
- But **what do we represent non-regular languages**?



Recap: Regular Languages & Proving Non-regularity

Problem 6 — Similar definitions, different languages

For some $k \geq 1$, define

$$B = \{1^k w \mid w \in \{0,1\}^* \text{ contains at least } k \text{ ones}\},$$

$$C = \{1^k w \mid w \in \{0,1\}^* \text{ contains at most } k \text{ ones}\}.$$

1. Find a simple regular expression for B.
2. Give a pumping-lemma proof that C is not regular.

▼ **Solution — click to expand**

For B, $k \geq 1$, and suppose we have a string $s \in B$, so $s = 1^k w$. Then,

$$s = 11^{k-1} w = 1(1^{k-1} w),$$

let $v = 1^{k-1} w$,, then we have $s = 1v$, which is a string in B with $k = 1$.

In other words, no matter what value k is, their corresponding strings are in the language B when $k = 1$.

Then let $k = 1$, we have $B={1w|w has least 1 one.}, so the corresponding regualr expression is

$$B = 1(0 \cup 1)^* 1(0 \cup 1)^*.$$

Assume C is regular and let p be its pumping length.

Choose $s = 1^p 01^p$. It belongs to C by taking $k = p$.

For every decomposition $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, we have $y = 1^t$ for some $t \geq 1$.

Pump down. The string $xz = 1^{p-t} 01^p$ has an initial run of only $p - t$ ones, while the remaining suffix contains at least p ones, i.e., in the pattern, the k is less than or equal to $p - t$.

For xz to be inside C, we need to find a k that makes $k > p$, but $p > p - t$, so there is acontradiction, i.e., $xz \notin C$. Hence, C is not regular.



How to describe non-regular languages?

- Describing a language => knowing what's inside
- Take $\{0^n 1^n : n \geq 0\}$ as an example,
- What strings are contained?
- ε, 01, 0011, 000111, …
- What's the connection between them?
- assume we start from $w_0 = \varepsilon$, $w_1 = 0w_0 1 = 01$
- $w_2 = 0w_1 1 = 0011$, …
- **We put a zero before and a one after the current string to create the next one → A rule for generating strings!**



A more powerful way to describe languages

Grammar: how to **generate** strings following a **finite** set of **rules**

- Variables: Symbols representing the **dynamically** generated strings
- Terminals: symbols only appearing on the right-hand side of the rules
- Rules: used for substitution, "variable -> string of variables and terminals"
- Start variable

➢ Example: $\{0^n 1^n : n \geq 0\}$,

- start variable: S
- $S \to \varepsilon$
- and then "**put a zero before and a one after the current string**"
- $S \to 0S1$
- $S \to 0S1 \mid \varepsilon$



Grammar

- a rule: "Variable -> string of variables and terminals"
- **When we observe a variable, we could replace it with the right-hand-side (RHS) string.**
- What if the left-hand-side is not only a variable?
- e.g., $0S \rightarrow 0S1$
- The substitution depends on neighborhoods, or more formally, **context**
- **Context-Free Grammars**: The rules work **regardless the context**.
- Context-Sensitive Grammars: The rules work depending on the context.



Context-Free Grammars (CFGs)

A CFG G is a 4-tuple vector (V, Σ, R, S):

- V: a finite set of variables
- Σ: a finite set of **terminal** symbols
- R: a finite set of rules (rule form: $V \rightarrow (V \cup \Sigma)^*$)
- S: Start variable

- Example:
- $V = \{S\}$
- $\Sigma = \{0,1\}$
- $R = \{S \rightarrow 0S1 \mid \varepsilon\}$
- $S = S$



CFG example

- $S \rightarrow RR$
- $R \rightarrow 0R1$
- $R \rightarrow \varepsilon$
- what strings are described using these rules?
- $0101, 010011,$ … but not 010101
- Understand $S \rightarrow RR$ as replacing S with two strings generated using R, while the two R have their independent generation process, i.e., the two R can be different



CFG example

- $L = \{all\ balanced\ parentheses\}$
- balanced parentheses: the left parentheses matches the right parentheses one-by-one following the order from left to right,
- like $()$, $(())$, $()()$, but not $)($.
- How to represent L using CFGs?
- Start by the empty string: $S \to \varepsilon$
- Given $()$, $(())$, we could add a left parenthesis to the left and a right parenthesis to the right, i.e., $S \to (S)$
- Given $()()$, we could also concatenate two balanced parentheses together, i.e., $S \to SS$

$$S \to \varepsilon \mid (S) \mid SS$$



CFG Derivation and Context-Free Language (CFL)

- Example: $S \rightarrow 0S1 \mid \varepsilon$
- Following the rules, we could have $S \Rightarrow 0S1 \Rightarrow 01$ or $S \Rightarrow \varepsilon$
- Such process is called **derivation**, applying rules to get **only terminals**
- $u \Rightarrow v$: u can change to v with one substitution step
- $u \overset{*}{\Rightarrow} v$: u can change to v with **some** substitution steps
- **Note: Distinguish $\rightarrow$ and $\Rightarrow$**
- **One used in rules, one used in derivation!**

- The corresponding language of a CFG is called **context-free language (CFL).**

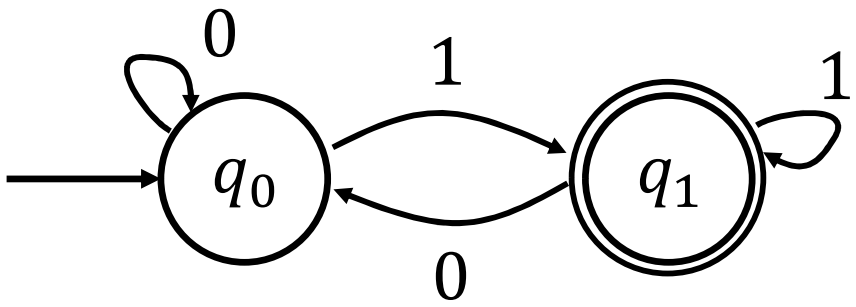
> Defn: A language A is a CFL if $A = L(G)$ for some CFG G.

- $L(G) = \{w | w \in \Sigma^* \; and \; S \overset{*}{\Rightarrow} w\}$



Relations between CFG, CFL and Regular Languages

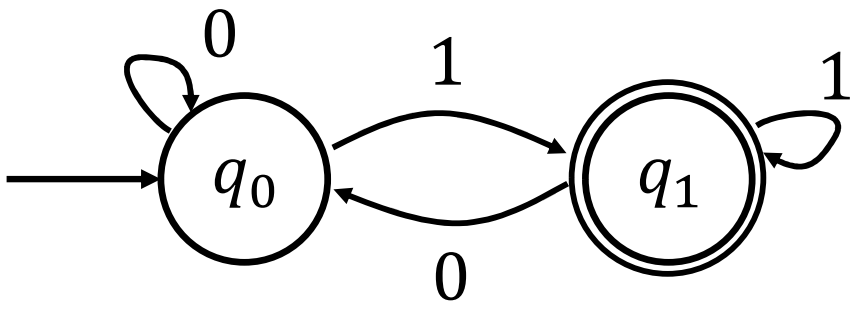
- Regular Languages can be represented as FAs or Regexes, and they are all equivalent. Let's take a DFA as an example.



- Assume we still start from the empty string ε, we could still generate strings following the FA **=> rules are just like transitions**
- $\delta(q_0, q_0) = 0$, when we encounter 0, we stay at q_0; similarly, if we add a 0, we stay at q_0 => $Aq_0 \rightarrow 0Aq_0$
- Note that here we generate strings **from left to right**, so the generated character should be **added before** the next generated character.

Relations between CFG, CFL and Regular Languages

- Regular Languages can be represented as FAs or Regexes, and they are all equivalent. Let's take a DFA as an example.



- when in q_0, if we add a 1, we transition to q_1 => $Aq_0 \rightarrow 1Aq_1$
- When in q_1, if we add a 0, we transition to q_0 => $Aq_1 \rightarrow 0Aq_0$
- When in q_1, if we add a 1, we stay at q_1 => $Aq_1 \rightarrow 1Aq_1$
- For the accepting state, $Aq_1 \rightarrow \varepsilon$
- **All regular languages have a corresponding CFG**
- **All regular languages are CFLs.**

Closure Properties of CFLs

- Recall: Regular languages are closed under regular operations.
- **Are CFLs also closed under regular operations?**
- **Union:**
- L_1 and L_2 are CFLs, and they have their corresponding CFGs G_1 and G_2
- for $L_1 \cup L_2$, we assume its start state to be S, and add a rule $S \to S_1 \mid S_2$, where S_1 and S_2 are the original start states of G_1 and G_2
- Then $L_1 \cup L_2$ also has a corresponding CFG => still CFL => Closed.

Theorem: if A and B are both context-free languages, so is $A \cup B$.

Equivalently, ***context-free languages are closed under union*.**



Closure Properties of CFLs

- L_1 and L_2 are CFLs, and they have their corresponding CFGs G_1 and G_2
- $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2)$
- The CFG we constructed for $L_1 \cup L_2$:
- $S = S$
- $V = V_1 \cup V_2 \cup \{S\}$
- $\Sigma = \Sigma_1 \cup \Sigma_2$
- $R = R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}$

Theorem: if A and B are both context-free languages, so is $A \cup B$.

Equivalently, ***context-free languages are closed under union*.**



Closure Properties of CFLs

- Recall: Regular languages are closed under regular operations.
- **Are CFLs also closed under regular operations?**
- **Concatenation:**
- L_1 and L_2 are CFLs, and they have their corresponding CFGs G_1 and G_2
- for L_1L_2, we add a rule $S \rightarrow S_1S_2$
- Then L_1L_2 has a corresponding CFG => still CFL => Closed.

Theorem: if A and B are both context-free languages, so is $A \circ B$.

Equivalently, ***context-free languages are closed under concatenation.***



Closure Properties of CFLs

- L_1 and L_2 are CFLs, and they have their corresponding CFGs G_1 and G_2
- $G_1 = (V_1, \Sigma_1, R_1, S_1), G_2 = (V_2, \Sigma_2, R_2, S_2)$
- The CFG we constructed for $L_1 \cup L_2$:
- $S = S$
- $V = V_1 \cup V_2 \cup \{S\}$
- $\Sigma = \Sigma_1 \cup \Sigma_2$
- $R = R_1 \cup R_2 \cup \{S \to S_1 S_2\}$

Theorem: if A and B are both context-free languages, so is $A \circ B$.

Equivalently, ***context-free languages are closed under concatenation.***



Closure Properties of CFLs

- Recall: Regular languages are closed under regular operations.
- **Are CFLs also closed under regular operations?**
- **Star:**
- L_1 is a CFL, and its CFG is G_1
- for L_1^*, it contains all the possible repetition of L_1, so informally we could imagine that the generated strings will be based on derivation of $S_1, S_1S_1, S_1S_1S_1$, …, where each S_1 generates the strings in L_1
- so we add a rule $S \rightarrow \varepsilon \mid S_1S$
- Then L_1^* has a corresponding CFG => still CFL => Closed.

Theorem: if A is a context-free language, so is A^*.

Equivalently, ***context-free languages are closed under star*.**



Closure Properties of CFLs

- L_1 is a CFL, and the corresponding CFG is G_1
- $G_1 = (V_1, \Sigma_1, R_1, S_1)$
- The CFG we constructed for L_1^*:
- $S = S$
- $V = V_1 \cup \{S\}$
- $\Sigma = \Sigma_1$
- $R = R_1 \cup \{S \to S_1 S \mid \varepsilon\}$

Theorem: if A is a context-free language, so is A^*.

Equivalently, ***context-free languages are closed under star.***



Relations between CFGs and CFLs

- Question: Could two CFGs correspond to the same CFL?
- **Possible!**
- Examples: G_1 and G_2 contain the same set of strings.

$$G_1: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \times F \mid F$$
$$F \rightarrow (E) \mid a$$

$$G_2: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + E \mid E \times E \mid (E) \mid a$$



Relations between CFGs and CFLs

- Examples: G_1 and G_2 contain the same set of strings.

$$G_1: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \times F \mid F$$
$$F \rightarrow (E) \mid a$$

$$G_2: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + E \mid E \times E \mid (E) \mid a$$

- for the same string $a + a \times a$, they provide different generation
- for G_1, E computes based on T, T based on F, so T is computed first, the derivation is $E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow a + T \times F \Rightarrow a + F \times F \Rightarrow a + a \times F \Rightarrow a + a \times a$



Relations between CFGs and CFLs

$$G_1: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \times F \mid F$$
$$F \rightarrow (E) \mid a$$

- for string $a + a \times a$, the derivation of G_1 is

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow F + T \\ &\Rightarrow a + T \\ &\Rightarrow a + T \times F \\ &\Rightarrow a + F \times F \\ &\Rightarrow a + a \times F \\ &\Rightarrow a + a \times a. \end{aligned}$$

① Starting from the left hand side, always apply the rules to the leftmost variable

② Until we reach a terminal (i.e., "a" in the example), and then we move on

③ Repeat until we have no further variables

➢ **Left-most derivation**



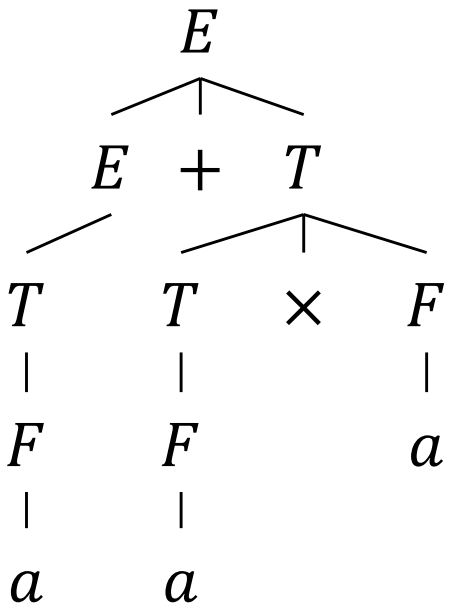
Relations between CFGs and CFLs

$$G_1: \Sigma = \{a, (,), +, \times\}$$
$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \times F \mid F$$
$$F \rightarrow (E) \mid a$$

- for string $a + a \times a$, the derivation of G_1 is

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow F + T \\ &\Rightarrow a + T \\ &\Rightarrow a + T \times F \\ &\Rightarrow a + F \times F \\ &\Rightarrow a + a \times F \\ &\Rightarrow a + a \times a. \end{aligned}$$

- Feels like a depth-first search?
- Represent the derivation of the string as a tree,



a parse tree

Relations between CFGs and CFLs

$$G_2: \Sigma = \{a, (,), +, \times\}$$
$$E \to E + E \mid E \times E \mid (\ E\ ) \mid a$$

- for string $a + a \times a$, the derivation of G_2 is

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow a + E \\ &\Rightarrow a + E \times E \\ &\Rightarrow a + a \times E \\ &\Rightarrow a + a \times a \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E \times E \\ &\Rightarrow E + E \times E \\ &\Rightarrow a + E \times E \\ &\Rightarrow a + a \times E \\ &\Rightarrow a + a \times a \end{aligned}$$

- In the set of rules of G_2, the priority of rule "$E \to E + E$" and "$E \to E \times E$" is the same!
- Just like NFAs, **non-deterministic**!

- The parsing trees of the two leftmost derivation are different and hence the string is derived ambiguously.
- We called such grammars **ambiguous.**



Review for today

- Grammars are more powerful tools to describe languages.
- Context-Free Grammars are those whose **rules only contain single variables in the left-hand side**.
- The language of CFGs are Context-Free Languages.
- Regular languages have their corresponding CFGs, and hence are all CFLs.
- The mapping between CFGs and CFLs is **many-to-one**.
- CFGs can be **ambiguous** if they can have **different derivation** for the same string.



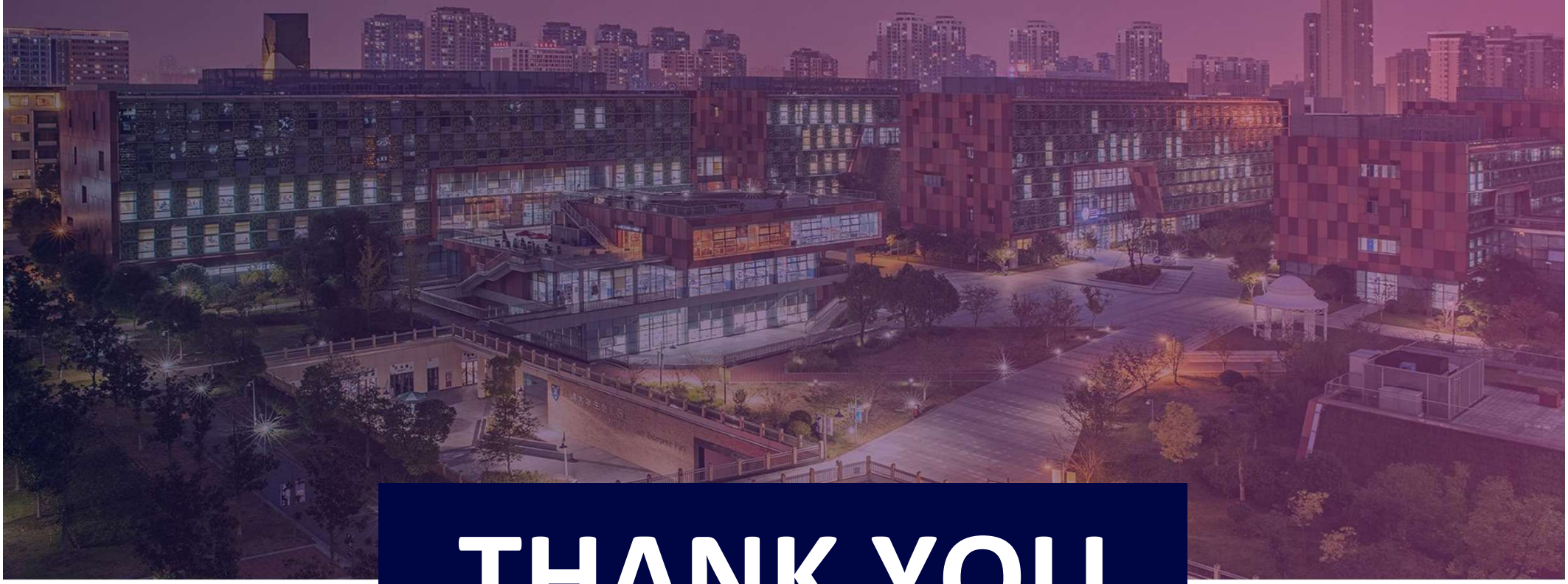
Q & A

① Suppose G_1 and G_2 have the same variables, the same terminals, the same rules, but different start variables, must $L(G_1) = L(G_2)$?

② If L is a CFL, and $L' \subseteq L$, is L' a CFL?

③ Can an unambiguous grammar have two different derivations of the same string?





THANK YOU

Xi'an Jiaotong-Liverpool University
西交利物浦大学

XJTLU | SCHOOL OF FILM AND TV ARTS